MODULE 2

Java Syntax and Core Constructs

The building blocks of every Java program — variables, operators, control flow, methods, and clean, idiomatic code style.


TEKsystems
Global Services
WORKFORCE
DEVELOPMENT

WHY JAVA SYNTAX MATTERS

Correct syntax is the foundation of every Java program — syntax is to programming what grammar is to a language.

- Java Syntax: the set of rules that define how Java code is written and structured. Without correct syntax, the compiler cannot understand your code.

WHY JAVA SYNTAX IS IMPORTANT

Communication with the Compiler

- Lets the compiler understand your intent and convert it into bytecode.

Prevents Errors Early

- Syntax errors are caught at compile time, before runtime failures.

Ensures Readability

- Well-structured syntax is easy to read, understand, and maintain.

Improves Reliability

- Correct syntax leads to predictable, robust behavior.

Foundation for Advanced Concepts

- OOP, collections, exceptions, concurrency — all built on correct syntax.

Boosts Professional Development

- Strong syntax skills build speed, confidence, and problem-solving ability.

KEY TAKEAWAY Syntax is not just about rules — it's about writing code that computers can understand and humans can maintain.

COMMON SYNTAX MISTAKES

The five mistakes that trip up almost every new Java developer.

- Missing semicolons (;)
- Incorrect use of brackets: {} [] ()
- Wrong keyword usage
- Case sensitivity — Java is case-sensitive
- Incorrect variable or method declarations

EXAMPLE — MISSING SEMICOLON

```
int x = 5    // missing semicolon → compile error
int x = 5;   // correct
```

KEY TAKEAWAY The compiler catches every one of these before your program ever runs — that's the whole point of syntax rules.

MODULE 2 LEARNING OBJECTIVES

By the end of this module, you will understand and apply Java syntax rules and core constructs to write effective programs.

1. Understand the basic structure of a Java program.
2. Declare variables and identify data types.
3. Use operators and expressions.
4. Control the flow of execution.
5. Define and use methods.
6. Organize code using packages and imports.
7. Perform basic input and output operations.
8. Write comments and follow code conventions.
9. Identify and avoid common syntax errors.
10. Combine core constructs to build simple programs.

KEY TAKEAWAY Strong syntax knowledge is the foundation for writing correct, maintainable programs and for advancing to OOP and beyond.

BUSINESS SCENARIO: EMPLOYEE REGISTRATION APPLICATION

You are a Java developer at TechSolutions Inc., building a console app to register employees.

- Business Context: HR wants a program that adds new employees, stores their information, and displays a summary of all registered employees.

Start Application → **Input Employee Details** → **Validate Data** → **Store Employee** → **Display All Employees**

OBJECTIVES OF THE APPLICATION

- Capture employee details from the user; store employee information in memory.
- Assign a unique employee ID automatically; display all registered employees.
- Ensure proper input validation; use clean code with correct Java syntax.

KEY FEATURES & CONCEPTS APPLIED (from Employee Registration)

- Menu-driven app (Add Employee, View All, Exit) with auto ID generation and input validation (email format, salary > 0, experience ≥ 0).
- Concepts you will apply: Variables & Data Types · Operators & Expressions · Control Flow · Methods · Arrays/Collections · Input/Output · Strings.

KEY TAKEAWAY This scenario runs through the rest of the module — every construct you learn will build toward this application.

Java Variables

A variable in Java is a **named memory location** used to store data of a specific type.

Types of Variables

1. Local Variable

- Declared inside a method, constructor or block.
- Scope is limited to that block.
- No default value.
- Stored in the **Stack**.

```java
public void show() {
    int age = 25;
    System.out.println(age);
}
```

Stack Memory

show()

age = 25

Created when the block/method is executed and destroyed when it exits.

2. Instance Variable

- Declared inside a class but outside any method.
- Belongs to an object.
- Has default values.
- Stored in the **Heap**.

```java
class Student {
  String name;  // instance variable
  int age;
}
```

Heap Memory

student1	student2
name = "John" age = 20	name = "Sara" age = 22

Created when the object is created and destroyed when the object is garbage collected.

3. Static (Class) Variable

- Declared with the `static` keyword inside a class.
- Belongs to the class, not to any object.
- Only one copy is shared by all objects.
- Stored in the **Method Area**.

```java
class Student {
  static int count = 0;  // static variable
}
```

Method Area

Class: Student

count = 0

Created when the class is loaded by the JVM and exists until the program ends.

Variable Declaration

Syntax:

```java
dataType variableName = value;
```

Examples:

```java
int age = 25;
double salary = 75000.50;
char grade = 'A';
boolean isJavaFun = true;
String name = "Alice";
```

Key Points

- Every variable must be declared with a data type.
- Variable names are case-sensitive.
- Local variables → Stack
Instance variables → Heap
Static variables → Method Area
- Use meaningful variable names to write readable code.

VARIABLE DECLARATION & MEMORY

Syntax, plus where variables and their data actually live.

```
dataType variableName = value;

int age = 25;
String name = "Anita";
```

MEMORY (CONCEPTUAL)

- Stack: age=25, salary=55000.75, isActive=true, name→(ref)
- Heap: the actual String object "Anita"

RULES FOR NAMING VARIABLES

- Must start with a letter, underscore (_), or dollar sign ($) — cannot start with a digit.
- Can contain letters, digits, _, or $; case-sensitive (age and Age are different).
- Cannot use Java reserved keywords; use meaningful names (employeeName, totalSalary).

WHY USE VARIABLES?

- A variable is a named memory location that stores data which can change during program execution.
- Two families you will meet next: primitive variables (int, double, char, boolean...) hold a value directly; reference variables (String, Scanner, Employee...) hold an address to an object.

KEY TAKEAWAY Best practices: choose appropriate types, initialize before use, keep variables in the smallest necessary scope.

PRIMITIVE DATA TYPES

Built-in data types in Java that store single values — not objects.

KEY CHARACTERISTICS

Single Values

Not objects.

Fixed Size & Range

Predictable memory use.

Stack Memory

Fast allocation.

Better Performance

Less overhead than objects.

WHEN TO CHOOSE WHICH TYPE

- int — default choice for whole numbers; long — only when values may exceed ~2.1 billion (remember the L suffix).
- double — default choice for decimals/money-style math; char — a single character; boolean — true/false flags.

KEY TAKEAWAY Numbers: int is preferred unless you need a specific range; use double for decimal calculations needing higher precision.

Java Primitive Data Types

Primitive data types are the basic building blocks in Java.
They hold **actual values** and are not objects.

Primitive Data Types

Numeric Types

Integer Types

	Size	Range (approx.)
byte	1 byte	-128 to 127
short	2 bytes	-32,768 to 32,767
int	4 bytes	-2^{31} to $2^{31}-1$ (-2,147,483,648 to 2,147,483,647)
long	8 bytes	-2^{63} to $2^{63}-1$ (approx.)

Floating-Point Types

	Size	Precision (approx.)
float	4 bytes	6–7 decimal digits
double	8 bytes	15–16 decimal digits

Non-Numeric Types

	Size	Description / Values
char	2 bytes	Holds a single Unicode character '\u0000' to '\uffff' (0 to 65,535)
boolean	1 bit	Represents logical values only **true** or **false**

Key Points

- Primitive types are stored in stack memory.
- They offer better performance and use less memory than objects.
- Primitive types have default values.

Default Values

- Numeric Types: 0 (or 0.0 for float/double)
- char: '\u0000' (null character)
- boolean: false

Java

Reference Data Types in Java

Reference types store a reference (address) to objects in the heap memory. Multiple references can point to the same object.

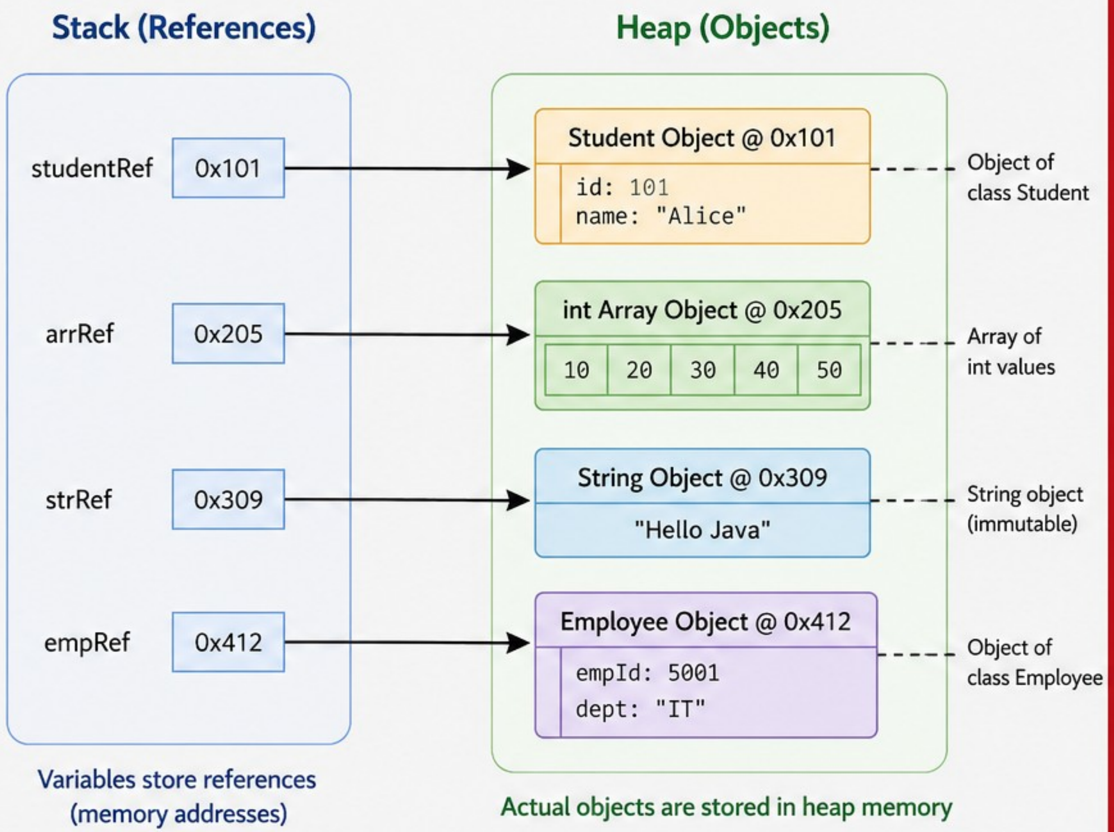
Types of Reference Data

Objects (Class Types)
Instances of user-defined classes

Arrays
Collection of elements of the same type

Strings
Sequence of characters (java.lang.String)

Interfaces
References to objects implementing an interface



Example

```java
Student studentRef = new Student();
int[] arrRef = {10, 20, 30};
String strRef = "Hello Java";
Employee empRef = new Employee();
```

Key Point: Reference variables do not store the actual data. They store the memory address of objects in the heap.

Java

JAVA REFERENCE TYPES

REFERENCE TYPES — EXAMPLE & BEST PRACTICES

A reference doesn't hold the object — it holds the object's address.

```
String name = "Anita";      // name refers to a String object
Employee emp = new Employee();  // emp refers to an Employee object
emp = null;                 // emp no longer points to any object
```

- Reference variables do not store the object itself, only the address; a reference can be reassigned.
- If a reference is set to null, it no longer points to any object; objects with no references become eligible for Garbage Collection.
- Best practices: always initialize references, check for null before use, prefer interfaces (List over ArrayList).

KEY TAKEAWAY Reference variables are addresses, not containers — understanding this prevents most null-pointer surprises.

Type Conversion and Casting in Java

Converting one data type to another.

Widening (Implicit) Conversion

- Automatically done by the Java compiler.
- Converts a smaller type to a larger type.
- No data loss.

Widening Hierarchy

byte (8-bit) → short (16-bit) → char (16-bit) → int (32-bit) → long (64-bit) → float (32-bit) → double (64-bit)

Example

```
int i = 100;
long l = i;        // int to long (implicit)
float f = l;       // long to float (implicit)
double d = f;      // float to double (implicit)
```

Narrowing (Explicit) Conversion (Casting)

- Must be done explicitly by the programmer.
- Converts a larger type to a smaller type.
- Possible data loss.

Narrowing Hierarchy (Reverse Direction)

double (64-bit) ↔ float (32-bit) ← long (64-bit) ← int (32-bit) ← char (16-bit) ← short (16-bit) ← byte (8-bit)

Example

```
double d = 123.456;
int i = (int) d;          // double to int (explicit)
long l = 1000L;
short s = (short) l;      // long to short (explicit)
```

Widening vs Narrowing

Aspect	Widening (Implicit)	Narrowing (Explicit)
Conversion	Smaller → Larger type	Larger → Smaller type
How	Automatic	Manual (Casting)
Data Loss	No	Possible
Example	int → long	long → int

Casting Syntax

```
(TargetType) expression
```

Example:

```
double d = 9.78;
int i = (int) d;
// i is 9
```

Important Notes

- Casting does not change the actual data, it only changes how it is interpreted.
- Be careful with narrowing conversions to avoid data loss.
- char can be converted to int implicitly, as char is treated as a 16-bit unsigned value.

Data Loss Example (Narrowing)

```
int x = 1000;
byte b = (byte) x; // x is 1000, but byte can store -128 to 127
// After conversion, b will be -24 (data loss)
```

int 1000 —(byte)→ byte -24

NARROWING, CASTING SYNTAX & COMMON ERRORS

(dataType) value — the explicit cast that tells the compiler you accept the risk.

```
double d = 9.78;
int n = (int) d;    // narrowing: n becomes 9, decimal part lost
char c = 65;        // special case: char holds the Unicode value 'A'
```

- Cannot convert from double to int without casting — the compiler requires (int) explicitly.
- Narrowing risks: possible loss of precision, and out-of-range values.
- Common runtime error: ArithmeticException, e.g. divide by zero.

KEY TAKEAWAY Widening conversions are safe and automatic; narrowing conversions require explicit casting and may lose data.

JAVA ARITHMETIC OPERATORS

Arithmetic operators are used to perform mathematical operations on numeric values.

Operator	Name	Description	Example	Result
+	Addition	Adds two operands	`int a = 10, b = 5;` `a + b`	15
−	Subtraction	Subtracts second operand from first	`int a = 10, b = 5;` `a - b`	5
*	Multiplication	Multiplies two operands	`int a = 10, b = 5;` `a * b`	50
/	Division	Divides first operand by second	`int a = 10, b = 5;` `a / b`	2
%	Modulus	Returns remainder of first operand divided by second	`int a = 10, b = 3;` `a % b`	1

EXAMPLE PROGRAM

```java
public class ArithmeticDemo {
   public static void main(String[] args) {
      int a = 10, b = 3;

      System.out.println("a + b = " + (a + b));
      System.out.println("a - b = " + (a - b));
      System.out.println("a * b = " + (a * b));
      System.out.println("a / b = " + (a / b));
      System.out.println("a % b = " + (a % b));
   }
}
```

OUTPUT

```
a + b = 13
a - b = 7
a * b = 30
a / b = 3
a % b = 1
```

KEY POINTS

- Work with numeric types: byte, short, int, long, float, double
- Division (/) between integers performs integer division
- Modulus (%) gives the remainder. Sign of result is same as dividend.
- Division by zero causes Arithmetic Exception
- Operator precedence applies: *, /, % before +, - (use parentheses)

PRECEDENCE ORDER (HIGH TO LOW)

* — / → % → + → −

Use parentheses ( ) to change the order.

ARITHMETIC OPERATORS — INTEGER VS FLOATING POINT

The same / operator behaves differently depending on operand types.

🔢 Integer Division

- int p = 7 / 2; → 3
- The decimal part is discarded entirely.

🔣 Floating Point Division

- double q = 7.0 / 2; → 3.5
- At least one operand must be a float/double.

- Division by zero with integers throws ArithmeticException; with doubles it produces Infinity or NaN.
- Use double (or float) for decimal results; modulus (%) works with both integers and floating-point numbers.

KEY TAKEAWAY Arithmetic operators are the foundation of calculations — knowing how they behave across data types prevents subtle bugs.

TRY IT YOURSELF — EXERCISE 1: CALCULATIONS

Reinforces: the arithmetic operators and precedence you just saw.

STEP	WHAT TO DO	COMMAND	RESULT
1	Create the file	Calculator.java	New Java class ready to edit
2	Compile	javac Calculator.java	Calculator.class produced
3	Run	java Calculator	Prompts for two numbers
4	Key concept	Use double, not int	Keeps decimal places in the quotient
5	Reference	exercises/exercise-01-calculator.md	Full starter code and steps

```
double a = Double.parseDouble(scanner.nextLine());  // first operand
double b = Double.parseDouble(scanner.nextLine());  // second operand
System.out.printf("Quotient: %.2f%n", a / b);       // double division keeps the decimal
```

TRY IT NOW Complete Exercise 1 before continuing — see exercises/exercise-01-calculator.md.

Relational Operators in Java

Relational operators are used to compare two values.
They return a boolean result: **true** or **false**.

Operator	Name	Meaning	Example (a = 10, b = 20)	Result
`==`	Equal to	Checks if two values are equal	`a == b`	false
`!=`	Not equal to	Checks if two values are not equal	`a != b`	true
`>`	Greater than	Checks if left value is greater than right value	`a > b`	false
`<`	Less than	Checks if left value is less than right value	`a < b`	true
`>=`	Greater than or equal to	Checks if left value is greater than or equal to right value	`a >= b`	false
`<=`	Less than or equal to	Checks if left value is less than or equal to right value	`a <= b`	true

Key Points

- Relational operators always return a boolean value (**true** or **false**).
- They are commonly used in decision making, loops, and conditional statements.

RELATIONAL OPERATORS — REAL-WORLD EXAMPLE

Employee Registration, using relational operators to gate business rules.

```
if (age >= 18) {          // allow registration
    registerEmployee();
}
if (salary > 50000) {     // apply tax calculation
    calculateTax();
}
if (id1 != id2) {         // avoid duplicate employee IDs
    createNewRecord();
}
```

String comparison: == compares references, not content, for String objects. Use .equals() to compare the text of two Strings, e.g. if (name1.equals(name2)).

KEY TAKEAWAY Relational operators are what turn business rules — age limits, salary thresholds, uniqueness checks — into working code.

LOGICAL OPERATORS

Logical operators in Java are used to combine boolean expressions and return a boolean result (true/false).

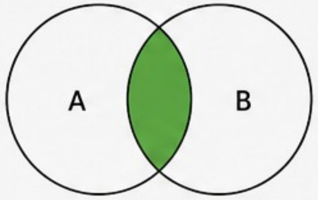
Operator	Name	Description	Example	Result
&&	Logical AND	Returns true only if both expressions are true.	(A > 5) && (B < 10)	true only if A > 5 and B < 10
||	Logical OR	Returns true if at least one of the expressions is true.	(A > 5) || (B < 10)	true if A > 5 or B < 10 or both
!	Logical NOT	Reverses the boolean value of the expression.	! (A > 5)	true if A <= 5

VISUAL REPRESENTATION

A && B

(Logical AND)

True only when both A and B are true.

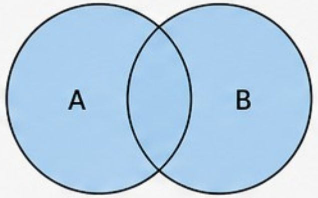


A	B	A && B
T	T	T
T	F	F
F	T	F
F	F	F

A || B

(Logical OR)

True when at least one of A or B is true.

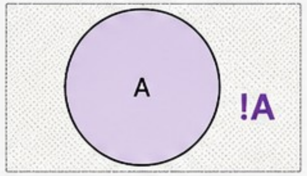


A	B	A || B
T	T	T
T	F	T
F	T	T
F	F	F

!A

(Logical NOT)

Reverses the value of A.



A	!A
T	F
F	T

Key Points

- Operands must be boolean (true/false) expressions.
- && has higher precedence than ||.
- ! has higher precedence than both && and ||.
- Short-circuit evaluation is applied in && and ||.

PRECEDENCE (High to Low)

! (NOT) > && (AND) > || (OR)

SHORT-CIRCUIT EVALUATION & REAL-WORLD USE

Java skips work it doesn't need to do — a performance and safety feature.

⏭ AND (&&)

- If the first operand is false, the second is never evaluated.

⏭ OR (||)

- If the first operand is true, the second is never evaluated.

```
if (employee != null && employee.getSalary() > 50000) {
    applyBonus(employee);   // safe: null check short-circuits first
}
```

Also — Logical XOR (^): true only when the two operands differ, e.g. true ^ true → false. XOR does not short-circuit — both sides are always evaluated.

KEY TAKEAWAY Logical operators let you build complex conditions from simple boolean expressions — essential for decision-making.

Assignment and Increment Operators

Operators used to assign values and update variables efficiently.

A+=1;

1. ASSIGNMENT OPERATORS

Used to assign a value to a variable.

Operator	Example	Equivalent To	Description
=	a = 10	a = 10	Simple assignment
+=	a += 5	a = a + 5	Add and assign
-=	a -= 5	a = a - 5	Subtract and assign
*=	a *= 5	a = a * 5	Multiply and assign
/=	a /= 5	a = a / 5	Divide and assign
%=	a %= 5	a = a % 5	Modulus and assign
&=	a &= 5	a = a & 5	Bitwise AND and assign
|=	a |= 5	a = a | 5	Bitwise OR and assign
^=	a ^= 5	a = a ^ 5	Bitwise XOR and assign
<<=	a <<= 2	a = a << 2	Left shift and assign
>>=	a >>= 2	a = a >> 2	Right shift and assign
>>>=	a >>>= 2	a = a >>> 2	Unsigned right shift and assign

Example:

```
int a = 10;
a += 5;   //  a = a + 5  =>  a = 15
a *= 2;   //  a = a * 2  =>  a = 30
a -= 8;   //  a = a - 8  =>  a = 22
```

2. INCREMENT AND DECREMENT OPERATORS

Used to increase or decrease the value of a variable by 1.

Operator	Name	Example	Explanation
++a	Pre-increment	`int a = 5;` `int b = ++a;`	• a is incremented to 6 • b gets 6
--a	Pre-decrement	`int a = 5;` `int b = --a;`	• a is decremented to 4 • b gets 4
Postfix (used after the variable)			
a++	Post-increment	`int a = 5;` `int b = a++;`	• b gets 5 (original value) • a is incremented to 6
a--	Post-decrement	`int a = 5;` `int b = a--;`	• b gets 5 (original value) • a is decremented to 4

Pre-increment: ++a

a 5 → (++a) → a 6 → Use 6

Increment first, then use

Post-increment: a++

a 5 → (a++) → Use 5 → a 6

Use first, then increment

Notes:

- Assignment operators provide a shorthand way to perform operations and assign the result.
- Use prefix (++a / --a) when you need the updated value immediately.
- Use postfix (a++ / a--) when you need the current value first, then update the variable.

INCREMENT & DECREMENT OPERATORS

++ and -- update a variable by 1 — but prefix and postfix behave differently.

```
int a = 5;
int b = ++a;    // prefix: increment first, then assign → a=6, b=6

int c = 5;
int d = c++;    // postfix: assign first, then increment → c=6, d=5
```

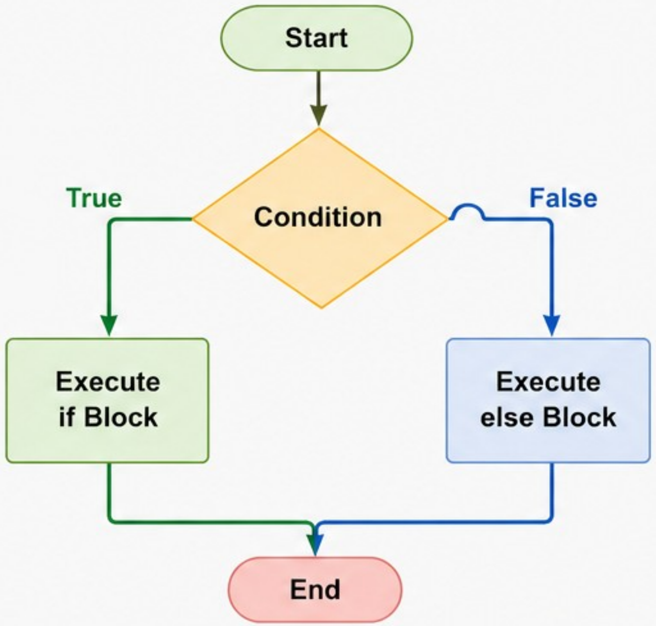
- Use prefix ++/-- when you need the updated value immediately; use postfix when you need the current value first.

KEY TAKEAWAY Understanding prefix vs postfix behavior is crucial — it's one of the most common sources of off-by-one bugs.

Decision Making with if and else in Java

The **if** statement executes a block of code if a condition is true.
The **else** statement executes a block of code if the condition is false.

Flowchart



Syntax

```
if (condition) {
   // code to be executed if condition is true
} else {
   // code to be executed if condition is false
}
```

Java Code Example

```java
public class IfElseDemo {

  public static void main(String[] args) {

      int num = 10;

      if (num >= 0) {
          System.out.println("Number is positive or zero");
      } else {
          System.out.println("Number is negative");
      }

  }

}
```

How it Works

- **Condition is true:**
  The `if` block is executed and the `else` block is skipped.
- **Condition is false:**
  The `else` block is executed and the `if` block is skipped.
- **Only one block** (either `if` or `else`) is executed.

IF…ELSE & IF…ELSE IF…ELSE LADDER

Two more shapes of decision-making, for two-way and multi-way branches.

2. IF…ELSE STATEMENT

```
if (condition) {
    // code if true
} else {
    // code if false
}
```

3. IF…ELSE IF…ELSE LADDER

```
if (marks >= 90) {          grade = "A";
} else if (marks >= 75) {   grade = "B";
} else {                     grade = "C";
}
```

KEY TAKEAWAY You can nest if inside another if for complex logic — but a ladder is usually clearer once you have 3+ conditions.

IF/ELSE RULES AND BEST PRACTICES

Core rules for writing correct if/else logic, plus where it shows up in business code.

- Conditions must be Boolean expressions — you cannot use an int or String directly as a condition.
- The first matching branch wins — only one block in an if/else if chain ever executes, evaluated top to bottom.
- Avoid deep nesting — three or more levels of nested if hurts readability; prefer a ladder or early return.
- Use switch instead of a long if/else if ladder when comparing one variable against many exact values.

REAL-WORLD EXAMPLES

Age Verification

Allow/deny based on age.

Discount Calculation

Tiered discount logic.

Grade Evaluation

Score ranges to letter grades.

KEY TAKEAWAY if and else help your program make smart decisions — they are the building blocks of logic in any application.

Switch Statements in Java

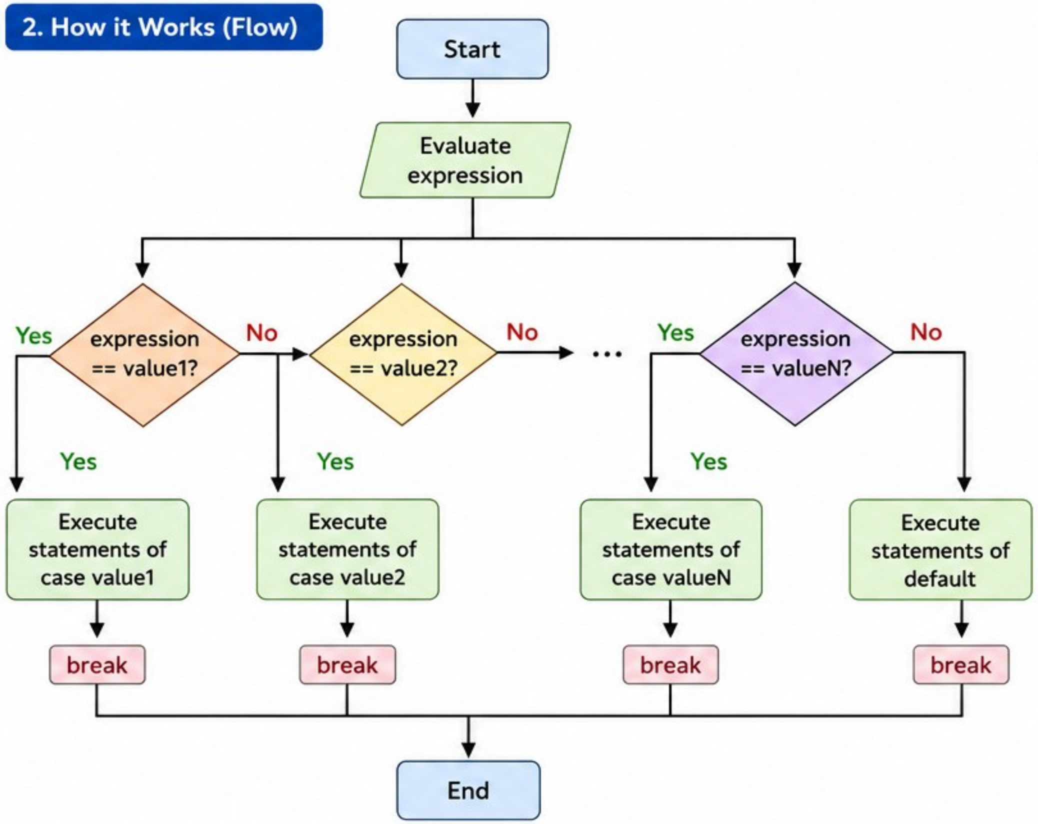
The **switch** statement allows a variable to be tested for equality against a list of values. It is an alternative to multiple `if-else if-else` statements.

1. Syntax

```
switch (expression) {
  case value1:
      // statements
      break;
  case value2:
      // statements
      break;
  ...
  case valueN:
      // statements
      break;
  default:
      // statements
}
```

- expression must be of type: byte, short, char, int, String, or enum.
- case values must be constant or literal and of the same type as expression.
- break prevents fall-through.
- default is optional and executes when no case matches.

2. How it Works (Flow)



3. Example

```
int day = 3;
switch (day) {
  case 1: System.out.println("Monday"); break;
  case 2: System.out.println("Tuesday"); break;
  case 3: System.out.println("Wednesday"); break;
  case 4: System.out.println("Thursday"); break;
  case 5: System.out.println("Friday"); break;
  default: System.out.println("Weekend");
}
```

Output

```
Wednesday
```

Key Points

- Improves readability over multiple if-else.
- Efficient for many discrete choices.
- Use break to avoid fall-through.
- Since Java 7, switch supports String.
- Since Java 14, supports arrow syntax (enhanced switch).

Enhanced Switch (Java 14+)

```
switch (day) {
  case 1 ->  System.out.println("Monday");
  case 2 ->  System.out.println("Tuesday");
  case 3 ->  System.out.println("Wednesday");
  default -> System.out.println("Weekend");
}
```

- No break needed.
- No fall-through.
- More concise and readable.

SWITCH — RULES, COMPARISON & USE CASES

When to reach for switch instead of a chain of if...else if.

- Case values must be constant or literal; duplicate case values are not allowed.
- default is optional but recommended; switch expressions (Java 14+) can return a value directly.

SWITCH	IF...ELSE IF LADDER
`Best for one variable, many exact values`	Best for ranges and complex boolean conditions

TWO SYNTAX FORMS

```
Traditional: case 1: ...; break;      Arrow (Java 14+): case 1 -> ...;  // no break needed
```

COMMON USE CASES

- Day/month names · Grade evaluation · Menu or command selection · Payment method handling · State/status codes.

KEY TAKEAWAY Use switch when comparing one variable against many possible values — it's more readable and less error-prone.

TRY IT YOURSELF — EXERCISE 2: DECISION MAKING

Reinforces: if/else and switch, as just covered.

STEP	WHAT TO DO	COMMAND	RESULT
1	Create the file	DecisionDemo.java	New Java class ready to edit
2	Compile	javac DecisionDemo.java	DecisionDemo.class produced
3	Run	java DecisionDemo	Prompts for a score and a day number
4	Key concept	switch (arrow form)	Matches one value against fixed cases — no break needed
5	Reference	exercises/exercise-02-decision-making.md	Full starter code and steps

```
if (score >= 90) { ... }                   // first true branch wins
else if (score >= 80) { ... }             // else-if chain checks top to bottom
switch (day) { case 1 -> ...; }             // arrow form: no fall-through, no break
```

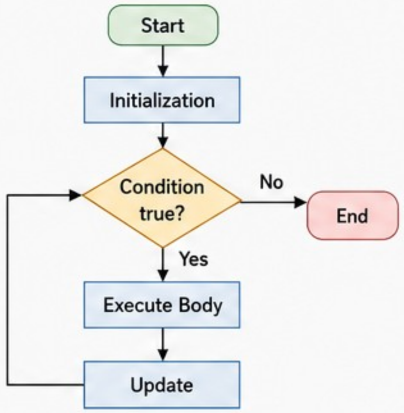
TRY IT NOW Complete Exercise 2 before continuing — see exercises/exercise-02-decision-making.md.

LOOPS IN JAVA

Loops are used to execute a block of code multiple times until a specific condition is met.

1. for LOOP

Used when the number of iterations is known.

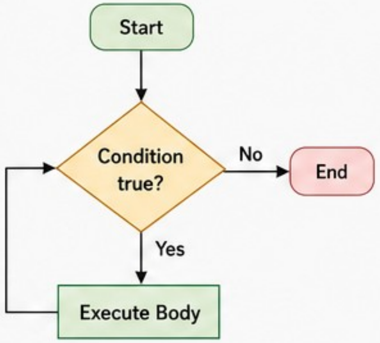



```
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

Use when iterations are count-based (e.g., for traversing arrays, tables).

2. while LOOP

Used when the number of iterations is not known in advance. Condition is checked before execution.

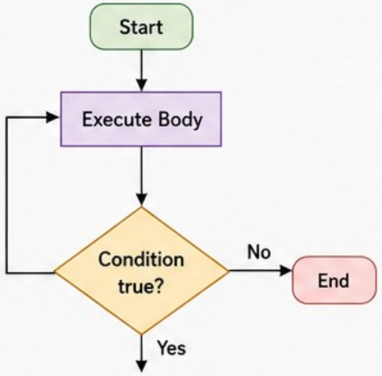



```
int i = 1;
while (i <= 5) {
    System.out.println(i);
    i++;
}
```

Use when we don't know how many times the loop will run.

3. do-while LOOP

Similar to while, but the condition is checked after execution. Executes at least once.

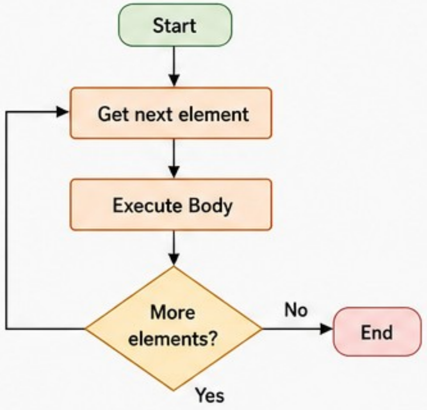



```
int i = 1;
do {
    System.out.println(i);
    i++;
} while (i <= 5);
```

Use when the loop body should execute at least once.

4. Enhanced for LOOP (for-each loop)

Used to iterate over arrays or collections easily.




```
int[] nums = {10, 20, 30, 40};
for (int n : nums) {
    System.out.println(n);
}
```

Use for clean and easy iteration over arrays and collections.

Key Points

- ✓ Use the right loop based on the requirement.
- ✓ Avoid infinite loops (ensure condition will eventually be false).
- Ensure loop variables are updated correctly.
- ✓ Prefer enhanced for loop for collections/arrays.
- ✓ Break and continue can be used to control loop flow.
- ✓ Write readable and maintainable loop logic.

Loop Comparison

Loop Type	Condition Check	Executes At Least Once?
for	Before each iteration	No
while	Before each iteration	No
do-while	After each iteration	Yes
Enhanced for	Before each iteration	No

BREAK, CONTINUE & COMMON LOOP TASKS

Two keywords that change loop flow mid-iteration.

```java
for (int i = 0; i < 10; i++) {
    if (i == 5) break;       // exits the loop entirely
    if (i % 2 == 0) continue; // skips to the next iteration
    System.out.println(i);
}
```

- Common loop tasks: calculate sum of first N numbers, find factorial, search an element in an array, process all elements in a collection, generate patterns.

KEY TAKEAWAY break exits the loop; continue skips only the current iteration — knowing the difference avoids logic bugs.

FOR LOOP IN JAVA

The for loop is used to repeat a block of code a specific number of times.

Syntax

```java
for (initialization; condition; update) {
    // body of the loop
}
```

How it Works

1. **Initialization:** Executed once at the beginning.
2. **Condition:** Checked before each iteration.
   If true, the loop body executes.
3. **Update:** Executed after each iteration.
4. When condition becomes false, the loop terminates.

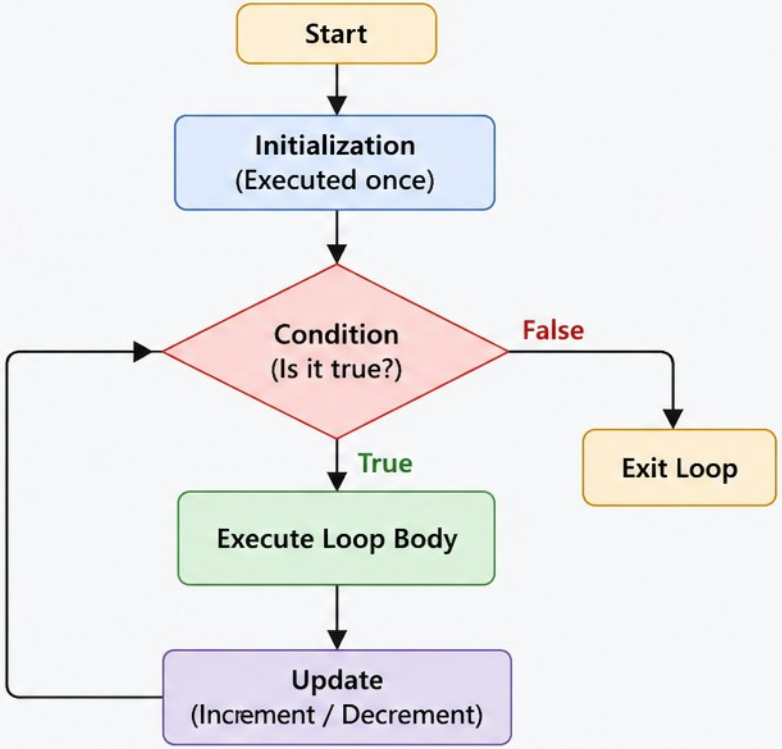
Example

```java
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

Output

```
1
2
3
4
5
```

Flow of Execution



Key Points

- **for** loop is ideal when the number of iterations is known.
- It combines initialization, condition checking, and update in a single line.
- Widely used in arrays, collections, and repetitive tasks.

```java
for (int i = 0; i < n; i++)
```

Initialization: `int i = 0` · Condition: `i < n` · Update: `i++`

NESTED FOR LOOP & BEST PRACTICES

A loop inside a loop — common for grids, tables, and patterns.

```
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        System.out.print(i * j + " ");
    }
    System.out.println();
}
```

- Use meaningful variable names; ensure the condition will eventually become false.
- Update the loop control variable correctly; use for loops when the iteration count is known.

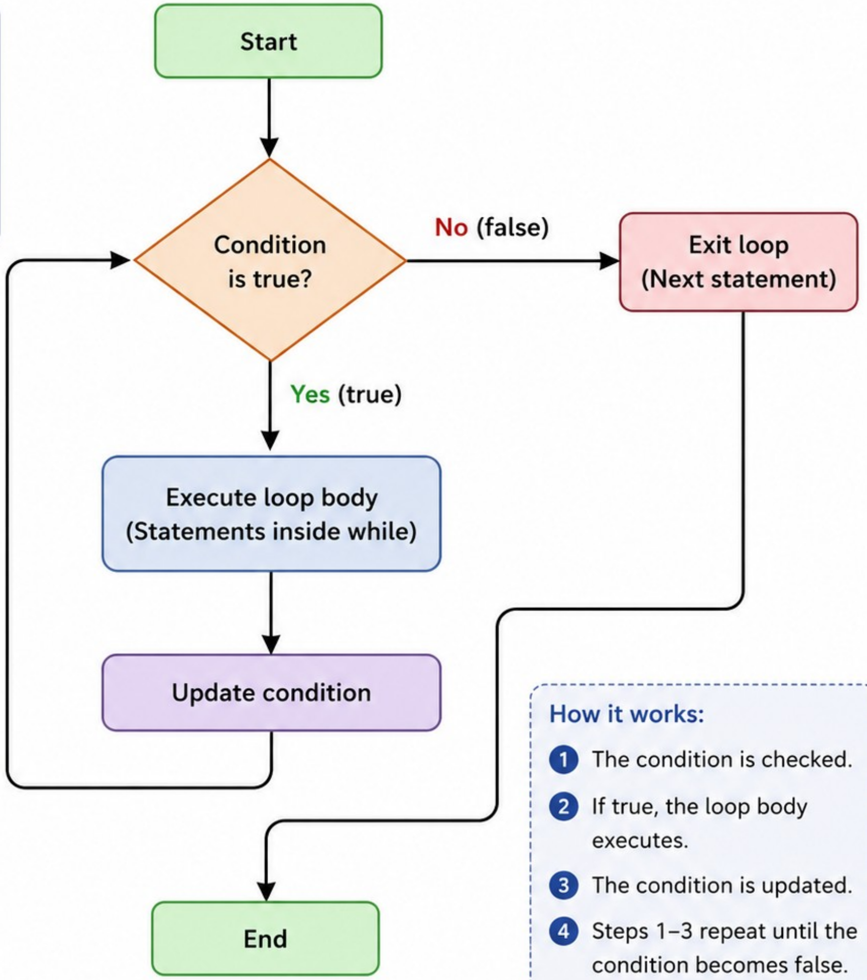
KEY TAKEAWAY The for loop is ideal when you know how many times the loop should run — compact, readable, and efficient.

WHILE LOOP IN JAVA

The **while** loop repeatedly executes a block of code as long as the given condition is true. When the condition becomes false, the loop stops

Syntax:

```
while (condition) {
   // body of loop
   // statements
}
```



Example:

```
int i = 1;
while (i <= 5) {
  System.out.println(i);
  i++;
}
```

Output:

```
1
2
3
4
5
```

How it works:

1. The condition is checked.
2. If true, the loop body executes.
3. The condition is updated.
4. Steps 1–3 repeat until the condition becomes false.

WHILE LOOP — WORKED EXAMPLE & USE CASES

Summing the first N numbers, the classic while-loop exercise.

```
int n = 5, sum = 0, i = 1;
while (i <= n) {
    sum += i;
    i++;
}
// sum = 15 (1+2+3+4+5)
```

- Common use cases: reading user input until valid, processing records until end of file, retrying operations until success, game loops, menu-driven programs.

KEY TAKEAWAY Use break to exit a while loop early when needed, but keep exit conditions explicit wherever possible.

do-while Loop in Java

The do-while loop is an exit-controlled loop. The block of code is executed at least once, then the condition is checked. If the condition is true, the loop repeats.

Syntax

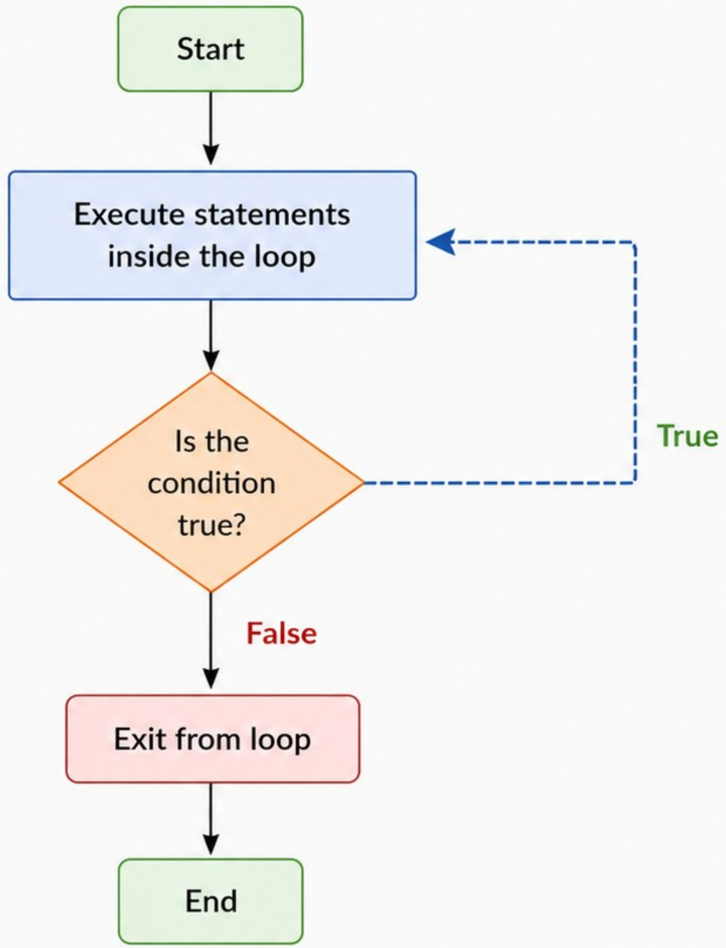
```java
do {
   // statements
} while (condition);
```

Example

```java
int i = 1;
do {
   System.out.println(i);
   i++;
} while (i <= 5);
```

Output

```
1
2
3
4
5
```



Key Point

In **do-while loop**, the statements are executed first, then the condition is checked. Therefore, the loop body executes at **least once**.

DO-WHILE — MENU-DRIVEN EXAMPLE

The menu is displayed at least once, even if the user exits immediately.

```
int choice;
do {
    System.out.println("1) Add  2) View  3) Exit");
    choice = scanner.nextInt();
    // handle choice...
} while (choice != 3);
```

- Best practices: initialize the loop control variable, ensure the condition will eventually become false, update it correctly.

KEY TAKEAWAY The do-while loop is perfect when you need the code to run at least once, then repeat based on a condition.

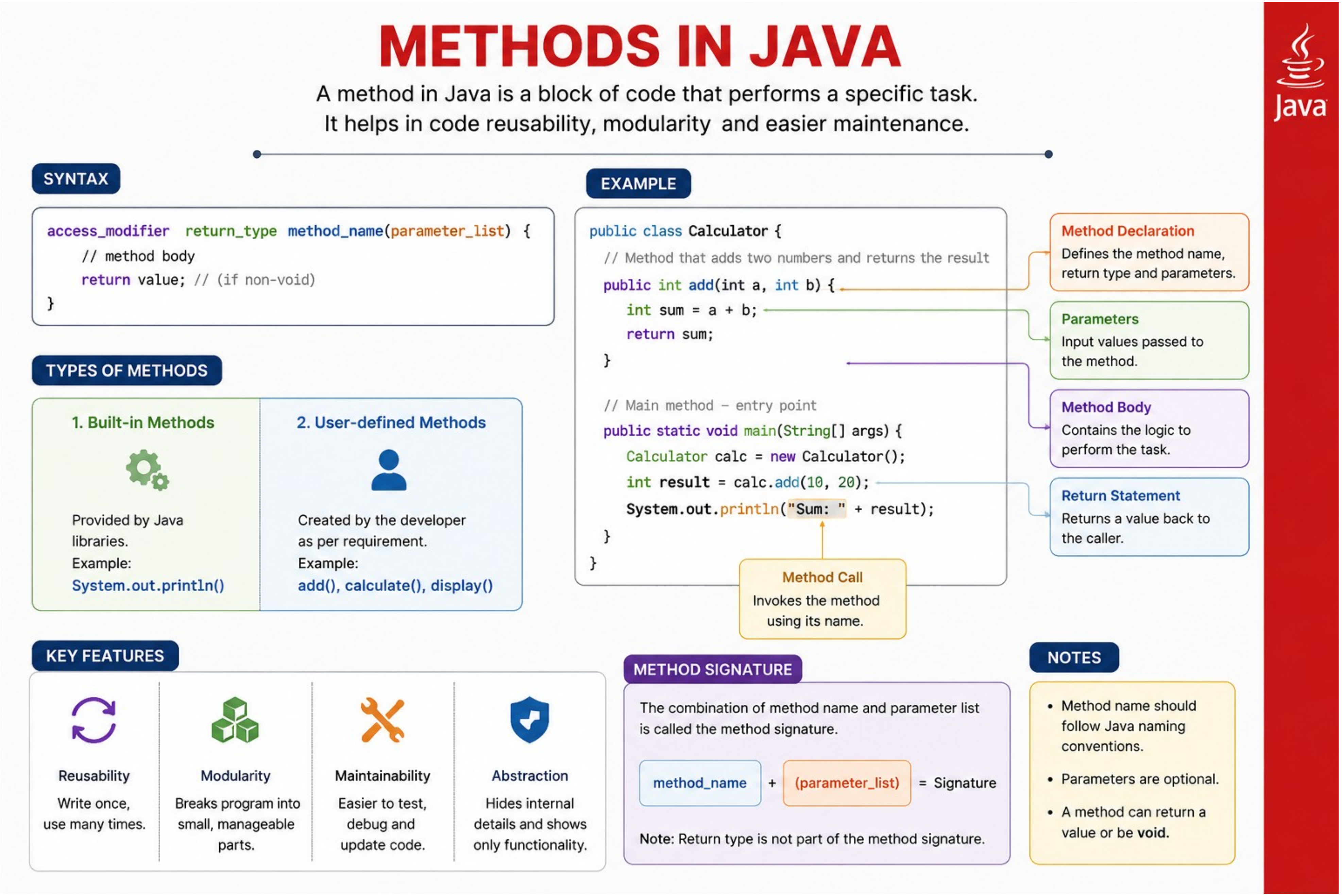
TRY IT YOURSELF — EXERCISE 3: LOOPS

Reinforces: for, while, and do-while, as just covered.

STEP	WHAT TO DO	COMMAND	RESULT
1	Create the file	LoopsDemo.java	New Java class ready to edit
2	Compile	javac LoopsDemo.java	LoopsDemo.class produced
3	Run	java LoopsDemo	Prints a table, a countdown, then a repeating menu
4	Key concept	do-while runs its body once first	Unlike while, which checks the condition before any pass
5	Reference	exercises/exercise-03-loops.md	Full starter code and steps

```
for (int i = 1; i <= 5; i++) { ... }         // known number of repetitions
while (count > 0) { count--; }               // condition checked before each pass
do { ... } while (choice.equals("menu"));    // body always runs at least once
```

TRY IT NOW Complete Exercise 3 before continuing — see exercises/exercise-03-loops.md.


METHODS IN JAVA
A method in Java is a block of code that performs a specific task.
It helps in code reusability, modularity and easier maintenance.
SYNTAX
access_modifier return_type method_name(parameter_list) {
// method body
return value; // (if non-void)
}
EXAMPLE
public class Calculator {
// Method that adds two numbers and returns the result
public int add(int a, int b) {
int sum = a + b;
return sum;
}
// Main method – entry point
public static void main(String[] args) {
Calculator calc = new Calculator();
int result = calc.add(10, 20);
System.out.println("Sum: " + result);
}
}
Method Declaration
Defines the method name, return type and parameters.
Parameters
Input values passed to the method.
Method Body
Contains the logic to perform the task.
Return Statement
Returns a value back to the caller.
Method Call
Invokes the method using its name.
TYPES OF METHODS
1. Built-in Methods
Provided by Java libraries.
Example:
System.out.println()
2. User-defined Methods
Created by the developer as per requirement.
Example:
add(), calculate(), display()
KEY FEATURES
Reusability
Write once, use many times.
Modularity
Breaks program into small, manageable parts.
Maintainability
Easier to test, debug and update code.
Abstraction
Hides internal details and shows only functionality.
METHOD SIGNATURE
The combination of method name and parameter list is called the method signature.
method_name + (parameter_list) = Signature
Note: Return type is not part of the method signature.
NOTES
Method name should follow Java naming conventions.
Parameters are optional.
A method can return a value or be void.
Java

METHODS — EXAMPLE, TYPES & BEST PRACTICES

A method that returns a value, and the two broad categories of methods.

```
public static int add(int a, int b) {
    return a + b;
}
int sum = add(3, 4);   // 7
```

Return a Value

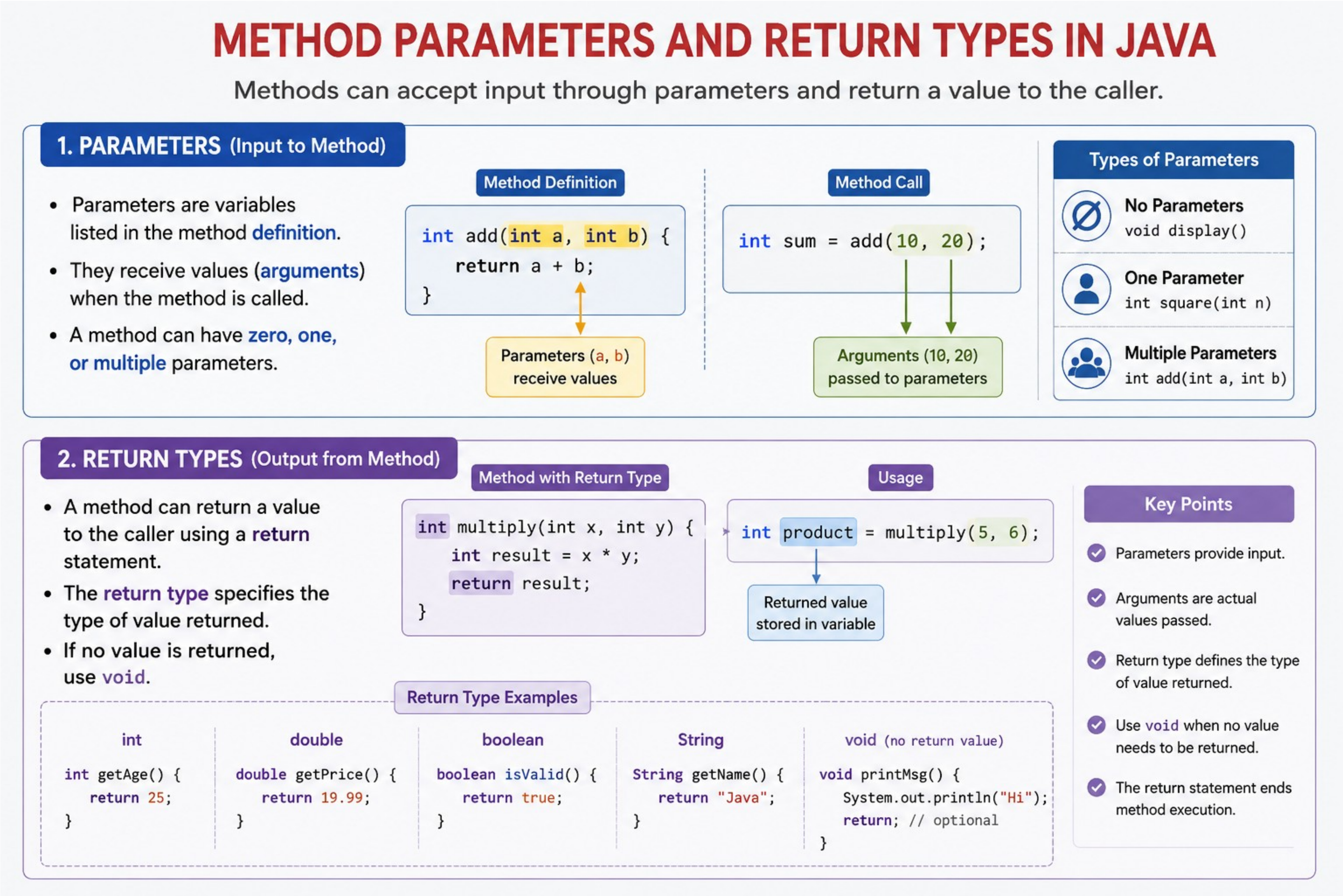
- Uses return to send a result back to the caller.

void (No Return)

- Performs an action, sends nothing back.

- Use meaningful method names; keep methods small and focused on one task.
- Use parameters and return values effectively; avoid code duplication.

KEY TAKEAWAY How methods work: the caller passes arguments, the method executes its body, and control returns to the caller.


METHOD PARAMETERS AND RETURN TYPES IN JAVA
Methods can accept input through parameters and return a value to the caller.
1. PARAMETERS (Input to Method)
Parameters are variables listed in the method definition.
They receive values (arguments) when the method is called.
A method can have zero, one, or multiple parameters.
Method Definition
int add(int a, int b) {
return a + b;
}
Parameters (a, b) receive values
Method Call
int sum = add(10, 20);
Arguments (10, 20) passed to parameters
Types of Parameters
No Parameters
void display()
One Parameter
int square(int n)
Multiple Parameters
int add(int a, int b)
2. RETURN TYPES (Output from Method)
A method can return a value to the caller using a return statement.
The return type specifies the type of value returned.
If no value is returned, use void.
Method with Return Type
int multiply(int x, int y) {
int result = x * y;
return result;
}
Usage
int product = multiply(5, 6);
Returned value stored in variable
Key Points
Parameters provide input.
Arguments are actual values passed.
Return type defines the type of value returned.
Use void when no value needs to be returned.
The return statement ends method execution.
Return Type Examples
int
int getAge() {
return 25;
}
double
double getPrice() {
return 19.99;
}
boolean
boolean isValid() {
return true;
}
String
String getName() {
return "Java";
}
void (no return value)
void printMsg() {
System.out.println("Hi");
return; // optional
}

PARAMETERS & RETURN — WORKED EXAMPLES

Three shapes: returning a value, returning nothing, and returning an object.

```java
// Example 1 — parameters + return
static int add(int a, int b) {
    return a + b;
}
```

```java
// Example 2 — void return
static void greet(String name) {
    System.out.println("Hi " + name);
}
```

```java
// Example 3 — returning an object
static Employee createEmployee(String name, double salary) {
    return new Employee(name, salary);
}
```

KEY TAKEAWAY Prefer returning a value over printing inside the method — it keeps methods reusable and testable.

PARAMETER PASSING — VALUE VS REFERENCE

Java is always pass-by-value — but for objects, the value being passed is a reference.

```
// Primitive — no effect on the original
static void change(int n) { n = n + 10; }
int x = 5; change(x);   // x is still 5

// Reference (object) — affects the original
static void change(int[] arr) { arr[0] = 99; }
int[] a = {1,2,3}; change(a);   // a[0] is now 99
```

KEY TAKEAWAY Reassigning a parameter inside a method never affects the caller's variable — but mutating the object it points to does.

METHOD OVERLOADING IN JAVA

Method Overloading is a feature in Java that allows a class to have multiple methods with the **same name** but **different parameters** (number, type or order of parameters).

HOW IT WORKS

- Methods must have the same name.
- Parameter list must be different.
- Return type can be same or different (but cannot be the only difference).

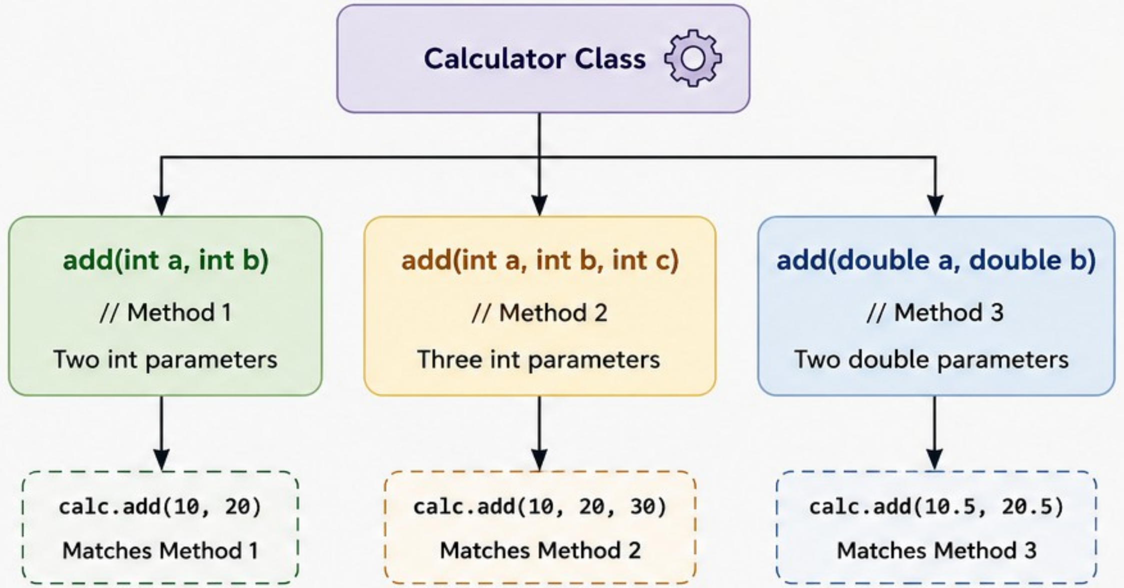
EXAMPLE

```java
class Calculator {

    // Method 1: two int parameters
    int add(int a, int b) {
        return a + b;
    }

    // Method 2: three int parameters
    int add(int a, int b, int c) {
        return a + b + c;
    }

    // Method 3: two double parameters
    double add(double a, double b) {
        return a + b;
    }
}
```

USAGE

```java
Calculator calc = new Calculator();
calc.add(10, 20);       // calls Method 1
calc.add(10, 20, 30);   // calls Method 2
calc.add(10.5, 20.5);   // calls Method 3
```

VISUAL REPRESENTATION



KEY POINTS

- Method overloading improves code readability.
- It is achieved at compile-time (static polymorphism).
- The compiler decides which method to call based on the arguments provided.
- Return type alone cannot be used for overloading.

NOT ALLOWED

```java
int add(int a, int b) { ... }
double add(int a, int b) { ... }
```

Same name and same parameters (with only return type different) → Compilation Error

In short: **Same name, different parameters → Method Overloading.**

HOW THE COMPILER CHOOSES AN OVERLOAD

Five-step resolution order, from exact match to varargs.

1. Exact match — parameter types match precisely.
2. Widening primitive conversion — e.g. int → long.
3. Widening reference conversion — e.g. subtype → supertype.
4. Autoboxing / unboxing — e.g. int → Integer.
5. Varargs — matched only as a last resort.

- Best practices: overload when methods perform the same task with different data, keep parameter lists simple, avoid too many variations.

KEY TAKEAWAY Method overloading gives flexibility by allowing multiple methods with the same name, resolved at compile time.

TRY IT YOURSELF — EXERCISE 4: METHODS

Reinforces: parameters, return values, and overloading.

STEP	WHAT TO DO	COMMAND	RESULT
1	Create the file	MethodsDemo.java	New Java class ready to edit
2	Compile	javac MethodsDemo.java	MethodsDemo.class produced
3	Run	java MethodsDemo	Prints square(4) and square(2.5)
4	Key concept	Overloading	Same method name, different parameter types
5	Reference	exercises/exercise-04-methods.md	Full starter code and steps

```
public static int square(int n) {           // int version
    return n * n; }                         // overload comes next
public static double square(double n) {...}  // overload: compiler picks by argument type
```

TRY IT NOW Complete Exercise 4 before continuing — see exercises/exercise-04-methods.md.

PACKAGES IN JAVA

Packages are used to group related classes and interfaces.
They help in organization, avoid naming conflicts, and control access.

PACKAGE STRUCTURE

```
src
├── com
│   └── example
│       ├── Student.java
│       └── Course.java
└── util
    ├── Helper.java
    └── Constants.java
```

PACKAGE DECLARATION

```java
// File: Student.java
package com.example;
public class Student {
    int id;
    String name;
}
```

HOW TO USE

```java
import com.example.Student;
public class Main {
    public static void main(String[] args) {
        Student s = new Student();
    }
}
```

WHY USE PACKAGES?

- ✓ Avoids name collisions (same class names).
- ✓ Organizes related classes.
- ✓ Provides access protection.
- ✓ Easier maintenance and reusability.

ACCESS LEVELS

Modifier	Within Same Package	Outside Package (Subclass)	Outside Package (Non-Subclass)
public	✓	✓	✓
protected	✓	✓	✗
default (no modifier)	✓	✗	✗
private	✓	✗	✗

TYPES OF PACKAGES

1. Built-in Packages (predefined)

- `java.lang` – core classes (String, Math, System)
- `java.util` – utilities (List, Set, Date)
- `java.io` – input/output classes
- `java.time` – date and time API

2. User-defined Packages

Created by developers to organize application-specific classes.

KEY POINTS

- First statement in a Java file must be the package declaration.
- Folder structure must match the package name. (com.example → com/example)
- Use import to access classes from another package.
- Packages help in encapsulation and access control.

PACKAGE TYPES & BUILT-IN PACKAGES

Two kinds of packages, and the ones you'll use every day.

Built-in Packages

- Provided by the JDK: java.lang, java.util, java.io.

User-Defined Packages

- Created by developers to organize their own code.

PACKAGE	PURPOSE
`java.lang`	Core classes — String, Math, Object (auto-imported)
`java.util`	Collections, Scanner, Date, and utility classes
`java.io`	Input/output classes — files, streams

KEY TAKEAWAY Best practices: use meaningful, all-lowercase package names, organize by feature or layer, and avoid the default package.

Import Statements in Java

Import statements allow you to use classes and interfaces from other packages without writing their fully qualified names.

1. Why Import?

- Java classes are organized in packages.
- Import statements bring classes into your current file.
- Improves readability and reduces the need to use fully qualified names.

2. Types of Import

Single Type Import

```
import packageName.ClassName;
```

Imports a specific class or interface from a package.

On-Demand (Wildcard) Import

```
import packageName.*;
```

Imports all classes and interfaces from a package.

3. How It Works

Package: java.util

- ArrayList
- HashMap
- Date
- ...

Using Import

```
import java.util.ArrayList;
import java.util.*;

public class Example {
   public static void main(String[] args) {
      ArrayList<String> list = new ArrayList<>();
     list.add("Java");
   }
}
```

ArrayList is available without using java.util.ArrayList

All classes in java.util are available (due to *)

4. Key Points

- ✓ import statements must be placed at the top of the file, before the class declaration.
- ✓ The java.lang package is imported automatically by default.
- ✓ If two packages have classes with the same name, use the fully qualified name to avoid ambiguity.

Example: Without Import

```
public class Example {
   public static void main(String[] args) {
      java.util.ArrayList<String> list =
            new java.util.ArrayList<>();
   }
}
```

Without import, we must use the fully qualified class name.

VALID IMPORTS & BEST PRACTICES

Wildcard imports work, but specific imports keep your namespace predictable.

VALID	USE WITH CARE
`import java.util.Scanner;`	import java.util.*; (wildcard — can cause name conflicts)
`import java.util.List;`	import static java.lang.Math.*; (static wildcard)

- Import is not transitive — importing a class does not give you its dependencies automatically.
- Commonly used packages: java.util (Scanner, ArrayList), java.io (files/streams), java.time (dates).
- Best practices: import only what you need, prefer specific imports over wildcard, keep imports organized.

KEY TAKEAWAY Import statements help you use classes from other packages easily, improving readability and reducing complexity.

ORGANIZING JAVA PROJECTS

A common, scalable project structure (Maven/Gradle style)

PROJECT ROOT
(example: my-app)

```
my-app/
├─ .idea/            IDE settings (optional)
├─ .mvn/             Maven wrapper files
├─ mvnw              Maven Wrapper (Linux/Mac)
├─ mvnw.cmd          Maven Wrapper (Windows)
├─ pom.xml           Maven configuration
├─ src/
│  └─ main/
│     ├─ java/
│     │  └─ com/
│     │     └─ example/
│     │        └─ myapp/
│     │           ├─ App.java
│     │           └─ UserService.java
│     └─ resources/
│        ├─ application.properties
│        └─ logback.xml
├─ test/
│  └─ java/
│     └─ com/example/myapp/
│        └─ AppTest.java
│  resources/
├─ target/           Build output (generated)
├─ .gitignore        Git ignore rules
└─ README.md         Project documentation
```

PATH	PURPOSE
src/main/java	Java source code for the application.
src/main/resources	Configuration files, properties, XML, templates, static resources.
src/test/java	Unit and integration test source code.
src/test/resources	Test configuration and test resources.
target/	Compiled classes, packaged JAR/WAR. Generated at build time.
pom.xml	Maven Project Object Model – manages dependencies, plugins, build.
.mvn/, mvnw*	Maven Wrapper – ensures consistent Maven version for the project.
.gitignore	Files and folders to ignore in version control.
README.md	Project overview, setup instructions, and notes.

PACKAGE STRUCTURE (BEST PRACTICE)

com.example.myapp

- controller – REST / MVC controllers
- service – Business logic
- repository – Data access (DAO/Repo)
- model – Entities / DTOs
- util – Utilities / Helpers

BENEFITS

- Clear separation of concerns
- Easy to navigate and maintain
- Scalable for small to large applications
- Works well with Maven and Gradle
- Industry-standard convention

TIPS

- Keep your package names in reverse domain format: com.example.myapp
- Put environment-specific configs in resources (e.g., application-dev.properties)
- Write tests for your code and keep them under src/test/java
- Keep dependencies up to date and avoid unused ones

LEGEND

Folder · File · C Java Class

PACKAGE NAMING & BEST PRACTICES

Reverse-domain naming keeps your packages globally unique.

```
com.companyname.project.module
com.bankapp.loanmanagement   // all lowercase, reverse domain notation
```

- Follow standard naming (all lowercase, reverse domain); keep packages small and focused.
- Avoid deep nesting (more than 4 levels); never use the default package.
- Organize by feature or by layer (model, service, controller); use IDE templates or build tools to scaffold structure.

EMPLOYEE REGISTRATION APP — EXAMPLE STRUCTURE

```
src/
  com/techsolutions/app/
    Main.java
    model/Employee.java
    service/EmployeeService.java
```

KEY TAKEAWAY A good project structure is the foundation of a successful Java application — organize, follow conventions, stay consistent.

Console Input with Scanner (Java)

Scanner is a class in java.utiil package that is used to take input from the console (keyboard).

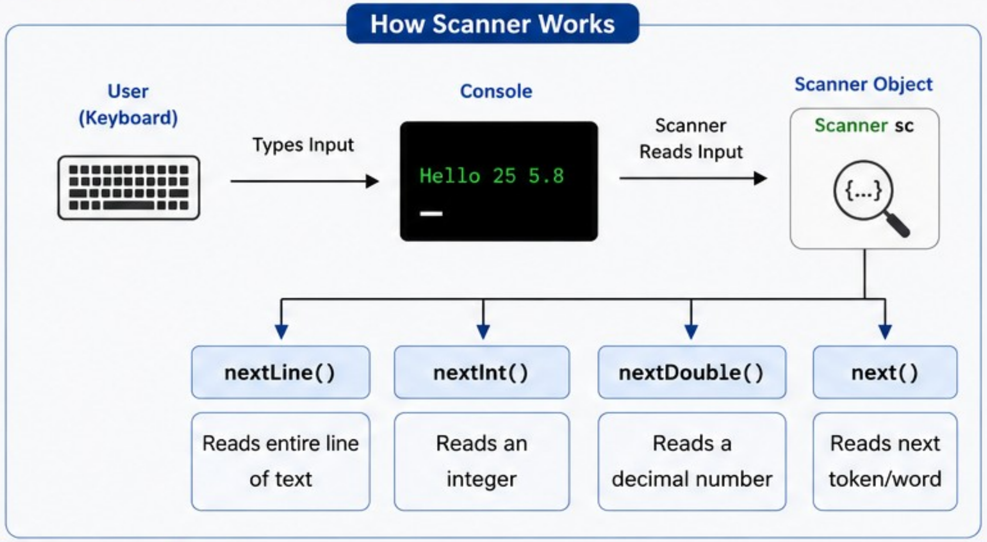
How it Works

1. Create a Scanner object and pass System.in to read input from the console.
2. Use nextXxx() methods to read different data types.
3. Scanner reads input token by token (separated by whitespace by default).
4. Close the Scanner object after use.

Example Code

```java
import java.util.Scanner;

public class ScannerInputDemo {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Reading different types of input
        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        System.out.print("Enter your age: ");
        int age = sc.nextInt();

        System.out.print("Enter your height: ");
        double height = sc.nextDouble();

        // Displaying input
        System.out.println("\nName    : " + name);
        System.out.println("Age     : " + age);
        System.out.println("Height  : " + height);

        sc.close();  // Always close the scanner
    }
}
```

How Scanner Works



Sample Run

Console Input (User types)

```
Enter your name: John Doe
Enter your age: 25
Enter your height: 5.8
```

Console Output (Program prints)

```
Name     : John Doe
Age      : 25
Height   : 5.8
```

Common Scanner Methods

Method	Description
nextLine()	Reads an entire line of text (including spaces)
next()	Reads next token/word (stops at whitespace)
nextInt()	Reads an integer
nextDouble()	Reads a double value
nextBoolean()	Reads a boolean value (true/false)
nextLong()	Reads a long value

Java

NEXT() VS NEXTLINE() & COMMON MISTAKES

The single most common Scanner bug in beginner Java code.

next()

- Reads only up to the first whitespace.
- Input: "John Doe" → Output: "John"

nextLine()

- Reads the entire line, including spaces.
- Input: "John Doe" → Output: "John Doe"

```
int age = sc.nextInt();
sc.nextLine();   // consume the leftover newline
String name = sc.nextLine();
```

KEY TAKEAWAY Mixing nextInt() and nextLine() without consuming the leftover newline is the #1 Scanner mistake — always chain a nextLine() after.

TRY IT YOURSELF — EXERCISE 5: PERSONAL DETAILS

Reinforces: next() vs nextLine() and the leftover-newline trap you just saw.

STEP	WHAT TO DO	COMMAND	RESULT
1	Create the file	PersonalDetails.java	New Java class ready to edit
2	Compile	javac PersonalDetails.java	PersonalDetails.class produced
3	Run	java PersonalDetails	Prompts for name, age, city
4	Key concept	nextInt() leaves \n in the buffer	Call scanner.nextLine() once to consume it
5	Reference	exercises/exercise-05-personal-details.md	Full starter code and steps

```
int age = scanner.nextInt();         // reads the int, leaves '\n' in the buffer
scanner.nextLine();                   // consume that leftover newline
String city = scanner.nextLine();     // now reads city correctly, not ""
```

TRY IT NOW Complete Exercise 5 before continuing — see exercises/exercise-05-personal-details.md.

TRY IT YOURSELF — EXERCISE 6: PRODUCT INFORMATION

Reinforces: reading mixed types safely with nextLine() + parse.

STEP	WHAT TO DO	COMMAND	RESULT
1	Create the file	ProductInfo.java	New Java class ready to edit
2	Compile	javac ProductInfo.java	ProductInfo.class produced
3	Run	java ProductInfo	Prompts for name, quantity, price
4	Key concept	Integer.parseInt(scanner.nextLine())	Avoids the nextInt() leftover-newline issue entirely
5	Reference	exercises/exercise-06-product-info.md	Full starter code and steps

```
String name = scanner.nextLine();                     // text may include spaces
int qty = Integer.parseInt(scanner.nextLine());        // read a line, then convert
double price = Double.parseDouble(scanner.nextLine());  // same pattern for decimals
```

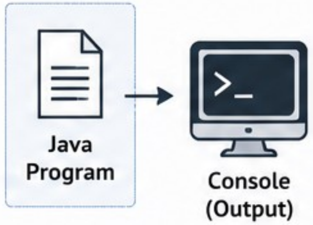
TRY IT NOW Complete Exercise 6 before continuing — see exercises/exercise-06-product-info.md.

Console Output and Formatting in Java

Java provides the `System.out` class to print output to the console.

1. Basic Output

- `System.out.print()` – prints without newline
- `System.out.println()` – prints with newline
- `System.out.printf()` – formatted output
- `System.out.format()` – formatted output (using Format specifiers)



2. printf() Formatting

```
int age = 25;
double pi = 3.14159;
String name = "Alice";

System.out.printf("Name: %s\n", name);
System.out.printf("Age: %d\n", age);
System.out.printf("PI: %.2f\n", pi);
```

Output

```
Name: Alice
Age: 25
PI: 3.14
```

Common Format Specifiers

Specifier	Meaning	Specifier	Meaning
%d	Integer (decimal)	%c	Character
%f	Floating point	%b	Boolean
%.2f	Floating point with 2 decimals	%x	Hexadecimal
%s	String	%%	Percent sign (%)

3. Formatting Examples

Width and Alignment

```
System.out.printf("|%10d|\n", 123);      // right aligned
System.out.printf("|%-10d|\n", 123);     // left aligned
System.out.printf("|%10s|\n", "Java");   // right aligned
System.out.printf("|%-10s|\n", "Java");  // left aligned
```

Output

```
|       123|
|123       |
|      Java|
|Java      |
```

10 is the minimum field width

Precision

```
System.out.printf("%.1f\n", 3.14159);
System.out.printf("%.3f\n", 3.14159);
System.out.printf("%.5f\n", 3.14159);
```

Output

```
3.1
3.142
3.14159
```

Controls the number of digits after decimal

Escape Sequences

```
System.out.println("Hello\nWorld");
System.out.println("Java\tProgramming");
System.out.println("Quote: \"Java\"");
System.out.println("Backslash: \\ ");
```

Output

```
Hello
World
Java    Programming
Quote: "Java"
Backslash: \\
```

Special characters for better formatting

4. Best Practices

- ✅ Use println() for complete lines and print() for inline output.
- ✅ Use printf() for readable and well-formatted output.
- ✅ Choose appropriate format specifiers for data types.
- ✅ Use \n for new lines and \t for tab spacing.

Note

printf() and format() return a PrintStream, so you can chain calls.

```
System.out.printf("Hello").printf(" World!");
```

Example: Student Details

```
String name = "Rahul";
int rollNo = 101;
double percentage = 87.65;

System.out.printf("%-10s  %5d  %6.2f%\n",
        name, rollNo, percentage);
```

Output

```
Rahul        101   87.65%
```

ESCAPE SEQUENCES & COMMON PITFALLS

Special characters, and the mistakes that produce garbled output.

ESCAPE	MEANING	EXAMPLE
`\n`	New line	"Line1\nLine2"
`\t`	Tab	"Name:\tAnita"
`\"`	Double quote	"She said \"Hi\""

- Common pitfalls: forgetting println() (output runs together), wrong format specifier for the data type.
- Mismatched specifiers and arguments (too many/too few %s), and not closing quotes.

KEY TAKEAWAY Using println(), printf(), and escape sequences together lets you build clean, well-formatted, user-friendly output.

TRY IT YOURSELF — EXERCISE 7: AREA OF CIRCLE

Reinforces: printf decimal formatting you just saw.

STEP	WHAT TO DO	COMMAND	RESULT
1	Create the file	CircleArea.java	New Java class ready to edit
2	Compile	javac CircleArea.java	CircleArea.class produced
3	Run	java CircleArea	Prompts for a radius
4	Key concept	Math.PI * r * r	Built-in constant for π, no import needed
5	Reference	exercises/exercise-07-circle-area.md	Full starter code and steps

```
double area = Math.PI * r * r;            // π × r²
System.out.printf("Area: %.2f%n", area);  // %.2f = two digits after the decimal
// try radius = 2.5                       // expect Area: 19.63
```

TRY IT NOW Complete Exercise 7 before continuing — see exercises/exercise-07-circle-area.md.

TRY IT YOURSELF — EXERCISE 8: BILL SUMMARY (CHALLENGE)

Reinforces: combining input, arithmetic, and money-style printf.

STEP	WHAT TO DO	COMMAND	RESULT
1	Create the file	BillSummary.java	New Java class ready to edit
2	Compile	javac BillSummary.java	BillSummary.class produced
3	Run	java BillSummary	Prompts for product, quantity, unit price
4	Key concept	%% inside printf	Prints a literal % (for "Discount (10%%)")
5	Reference	exercises/exercise-08-bill-summary.md	Full starter code and steps

```
double total = qty * price;             // before discount
double discount = total * 0.10;         // 10% off
System.out.printf("Discount (10%%): %.2f%n", discount);  // %% escapes the percent sign
```

TRY IT NOW Complete Exercise 8 before continuing — see exercises/exercise-08-bill-summary.md.

TRY IT YOURSELF — EXERCISE 9: PERSONAL PROFILE (BONUS)

Reinforces: printf width specifiers for aligned columns.

STEP	WHAT TO DO	COMMAND	RESULT
1	Create the file	PersonalProfile.java	New Java class ready to edit
2	Compile	javac PersonalProfile.java	PersonalProfile.class produced
3	Run	java PersonalProfile	Prompts for name, age, city, hobby
4	Key concept	%-12s left-aligns in a 12-char field	Same idea Lab 2's student list table uses
5	Reference	exercises/exercise-09-profile-bonus.md	Full starter code and steps

```
System.out.printf("%-12s | %-20s%n", "Field", "Value");  // header row
System.out.printf("%-12s | %-20s%n", "Name", name);      // left-aligned, 12-char field
System.out.println("-------------|---------------");     // simple divider line
```

TRY IT NOW Complete Exercise 9 before continuing — see exercises/exercise-09-profile-bonus.md.

Java Naming Conventions

Following standard naming conventions makes code readable, consistent, and easy to maintain.

Java

ELEMENT	CONVENTION	EXAMPLE

Class	• Use PascalCase
	• Noun or noun phrase
		CustomerAccount
		`public class CustomerAccount {`
		`    // class body`
		`}`

Interface	• Use PascalCase
	• Adjective or noun
		Runnable
		`public interface Runnable {`
		`    // interface body`
		`}`

Method	• Use camelCase
	• Verb or verb phrase
		calculateTotalAmount()
		`public double calculateTotalAmount() {`
		`    // method body`
		`}`

Variable (local)	• Use camelCase • Noun or noun phrase	**totalAmount** `double totalAmount = 0.0;`
Constant (static final)	• Use UPPER_SNAKE_CASE • Noun or noun phrase	**MAX_RETRY_COUNT** `public static final int MAX_RETRY_COUNT = 3;`
Package	• Use lowercase • Reverse domain name notation	**com.example.project** `// package com.example.project;`

GENERAL RULES

- Aa Use meaningful names
- Names should be readable and descriptive
- Avoid using single-letter names (except loop variables like i, j, k)
- 123 Do not start names with digit
- Do not use special characters (except _ and $)

BEST PRACTICES

- Be consistent within the project
- Follow standard conventions strictly
- Use IDE templates or code style settings
- Good names improve code readability and maintainability

Good Naming = Self-documenting Code | Follow conventions today, thank yourself tomorrow.

NAMING — GENERAL RULES, DO'S & DON'TS

Seven rules, and the mistakes to avoid.

- Names can contain letters, digits, $, and _ — but cannot start with a digit; names are case-sensitive.
- Do not use reserved keywords; names should be meaningful, descriptive, and pronounceable.
- Avoid abbreviations and unclear names; be consistent within the project.

Do's

- Meaningful names, camelCase for methods/vars, PascalCase for classes, UPPER_SNAKE_CASE for constants.

Don'ts

- Single-letter names (except generics), leading numbers, reserved keywords, inconsistency.

KEY TAKEAWAY Package naming uses reverse domain style: com.example.project.module — e.g. com.bankapp.loanmanagement.

CODE STYLE BEST PRACTICES IN JAVA
Good code style improves readability, maintainability and teamwork.
Java
GOOD STYLE • BETTER CODE • BETTER SOFTWARE
1
FOLLOW NAMING CONVENTIONS
Use meaningful and descriptive names.
Use camelCase for variables and methods.
Use PascalCase for classes.
Use UPPER_SNAKE_CASE for constants.
// Good
class Customer {
private String firstName;
private final int MAX_RETRY = 3;
public void calculateTotal() {
// ...
}
}
2
USE PROPER INDENTATION
Use 4 spaces per indent
Do not use tabs.
Keep code blocks consistent.
// Good
if (x > 0) {
for (int i = 0; i < n; i++) {
doWork(i);
}
} else {
handleError();
}
3
KEEP LINES SHORT AND READABLE
Limit line length to 100–120 characters.
Avoid long parameter lists or expressions.
Break long lines logically.
// Good
int total = price * quantity
+ tax - discount
+ shippingCharge;
Java
4
USE SPACES CONSISTENTLY
Use spaces around operators, keywords and after commas.
Do not use extra spaces unnecessarily.
// Good
if (a == b) {
int sum = a + b;
list.add(item);
}
// Avoid
if(a==b){
int sum=a+b;
list.add(item);
}
5
WRITE MEANINGFUL COMMENTS
Explain why, not what.
Keep comments up to date.
Do not over-comment obvious code.
// Calculate total after tax and discount
double total = price * (1 - discount) + tax;
6
ORGANIZE IMPORTS
Use specific imports.
Avoid wildcard imports (avoid importing *)
Group imports:
1. java.*
2. javax.*
3. third-party
4. project packages
// Good
import java.util.List;
import java.util.ArrayList;
import javax.sql.DataSource;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import com.myapp.model.User;
import com.myapp.service.UserService;
7
KEEP CODE SIMPLE AND CONSISTENT
Prefer simplicity over clever code.
Be consistent within the project.
Follow team or industry standards (e.g., Google Java Style Guide).
Readable Code
Happy Team
Fewer Bugs
Easy Maintenance

CODE STYLE — METHOD DESIGN, COMMENTS & CHECKLIST

The habits that separate professional code from a first draft.

- Method Design: keep methods small and focused; the method name should describe what it does; return early to reduce nesting.
- Comments: explain WHY, not WHAT; keep comments up-to-date; use Javadoc for public APIs.
- Organization: Fields → Constructors → Methods → Inner classes.

QUICK CHECKLIST

- Indent with 4 spaces, no tabs; use braces for all control statements; meaningful names for all identifiers.
- Keep lines within 120 characters; comments explain why, not what; keep methods small and focused.

KEY TAKEAWAY Follow best practices today for better code, better teams, and better software.

PRE-LAB EXERCISES OVERVIEW

Nine short console programs to complete before Lab 2, from core syntax practice to a bonus profile challenge.

- Exercise Goal: practice variables, Scanner input, arithmetic, and printf() output in small, focused programs before combining them in Lab 2.

EXERCISES

1. Calculations — read two numbers; show sum, difference, product, and quotient.
2. Decision Making — grade a score with if/else if/else; name a day with switch.
3. Loops — multiplication table with for; countdown with while; menu with do-while.
4. Methods — a square method overloaded for int and double parameters.
5. Personal Details — read name, age, and city; display a greeting.
6. Product Information — read product name, quantity, and price.
7. Area of Circle — read a radius; calculate area using Math.PI.
8. Bill Summary (challenge) — compute total, 10% discount, and final amount.
9. Personal Profile (bonus) — aligned printf table for name, age, city, and hobby.

KEY TAKEAWAY These exercises build every construct Lab 2 combines: variables, Scanner input, decisions, loops, methods, arithmetic, and printf() formatting.

LAB 2 TASKS, DELIVERABLES AND EVALUATION

Java Syntax and I/O — build a menu-driven Student Management console app.

BUILD REQUIREMENTS

Package Structure

com.academy.student folder layout.

Student Model

Fields, constructor, getters/setters.

Menu Loop

while (true) + switch, choices 1-5.

Add & Display

Validate input; printf table.

Search & Average

Find by ID; compute average marks.

Validation

Reject bad input, duplicate IDs, out-of-range marks.

EVALUATION WEIGHTS

Project Structure & Packages 15% · Student Model & Menu Loop 30% · Core Operations (Add/Display/Search/Average) 30% · Validation & Formatted Output 15% · Code Quality & Evidence 10%.

SUBMISSION ITEMS

Source under java-bootcamp/examples/Lab2-JavaSyntax/src/com/academy/student/ (Student.java, StudentManager.java, Main.java); screenshots of the menu, add, display, average, and exit flow; a short reflection note; and working compile/run commands.

KEY TAKEAWAY One console app, every construct: packages, Scanner, arrays, methods, loops, and validation working together.

MODULE 2 SUMMARY

You learned how to write basic Java programs that take input, calculate, and display formatted output.

- Java program structure and syntax; variables, data types, and operators.
- Taking input using the Scanner class; performing calculations; displaying formatted output.

CONCEPT	MEANING
`Class & main()`	Every program starts with a class; main() is the entry point
`Data Types`	int, double, String, boolean, char, ...
`Scanner Methods`	nextLine(), nextInt(), nextDouble()
`Output Methods`	print(), println(), printf()

KEY TAKEAWAY Real-world use: collecting user information, business calculations, reports and bills, interactive console programs.

MODULE 2 COMPLETION CHECKLIST & NEXT STEPS

Self-check before moving on to control statements and beyond.

- I can write a simple Java program with a class and main().
- I can declare variables and use different data types.
- I can take user input using Scanner.
- I can perform calculations.
- I can display formatted output using printf().
- I follow naming conventions and write clean code.

KEY REMINDERS FROM THIS MODULE

- Type conversion: widening (int→double) is automatic; narrowing needs an explicit cast, e.g. (int) d, and may lose data.
- Formatted output %d / %.2f / %s, and practice — write small programs and test with different inputs.

What's Next: Control Statements · Loops · Arrays and Strings · Bigger programs and projects.

KEY TAKEAWAY Every box checked here is a construct you'll use in every Java program you write from now on.

MODULE 2 KNOWLEDGE CHECK — MULTIPLE CHOICE (1–5)

Choose the best answer. Correct answers are marked ✓.

1. What is the correct form of the Java main() method?
A) void main(String[] args) **B) public static void main(String[] args) ✓** C) public void main() D) static main(String args)

2. Which of these is a valid Java variable name?
A) 2total **B) total_amount ✓** C) class D) total-amount

3. Which type stores decimal values such as 45.75?
A) int **B) double ✓** C) char D) boolean

4. Which statement correctly reads an integer with Scanner?
A) sc.nextInt() ✓ B) sc.readInt() C) sc.getInt() D) sc.int()

5. What is the output of int p = 7 / 2;?
A) 3.5 **B) 3 ✓** C) 4 D) 3.0

KEY TAKEAWAY Five more multiple-choice questions continue on the next slide.

MODULE 2 KNOWLEDGE CHECK — MULTIPLE CHOICE (6–10)

Choose the best answer. Correct answers are marked ✓.

6. Which method displays text without moving to a new line?
A) System.out.println() **B) System.out.print()** ✓ C) System.out.printf() D) System.out.write()

7. Which format specifier shows a value with two decimal places?
A) %d **B) %.2f** ✓ C) %s D) %2d

8. What is the result of 5 + 3 * 2 (operator precedence)?
A) 16 **B) 11** ✓ C) 13 D) 10

9. Given int x = 4, y = 2, what is x + y * x?
A) 24 **B) 12** ✓ C) 10 D) 8

10. Which type is correct for reading a full line of text, including spaces?
A) sc.next() **B) sc.nextLine()** ✓ C) sc.nextInt() D) sc.readLine()

KEY TAKEAWAY That’s the full ten-question knowledge check — next, the module’s key takeaways before you head into the lab.

Well done

You can now write, compile, and run Java programs using core syntax and constructs with confidence.

Object-oriented programming is next in your toolkit.